APS Failure Detection in Scania Trucks:

A Comparative Study of Machine Learning Classifiers

Marcus Giannini

CS 460 - Intro. to Applied Machine Learning

May 4, 2026

Abstract

This project compares several machine learning models for predicting Air Pressure System (APS) failures in Scania heavy trucks. At first, the task looks like a normal binary classification problem, but the dataset makes it more complicated. The training set has 60,000 rows, 170 feature columns, and only 1,000 positive APS-failure cases. That means a model can look accurate while still missing the cases that matter most. I cleaned the data by removing columns with more than 70% missing values, filling the remaining missing values with feature medians, scaling the numerical features, and using class weighting when the model supported it. I then tuned and compared six models: Logistic Regression, K-Nearest Neighbors, Decision Tree, Linear SVM, Random Forest, and XGBoost. XGBoost had the best balanced performance with an F1 score of 0.8864 and ROC-AUC near 0.995. However, the official Scania cost metric gives a much larger penalty to missed APS failures than to unnecessary inspections. Under that cost formula, Logistic Regression had the lowest total cost because it missed only 24 APS failures. So the main result is not that one model wins in every way. XGBoost is the strongest balanced classifier, while Logistic Regression is the better cost-based choice.

1. Introduction

1.1 Problem Definition

The goal of this project is to predict whether a truck failure is related to the Air Pressure System, or APS, instead of some other part of the truck. The APS matters because it supports things like braking and gear changes. The best way I understood the problem is that the model is being used as a maintenance warning tool. It is trying to catch trucks that may need APS-related attention before a larger failure happens.

This is a binary classification problem because each record is either negative or positive. The hard part is that the positive class is rare. Most rows are negative, so a model can get a high accuracy score by mostly predicting the negative class. That would not really solve the problem because the main point is to find the APS failures.

The cost structure also changes how the answer should be judged. A false positive costs 10 because it means a truck gets an unnecessary inspection. A false negative costs 500 because the model missed a real APS failure (Scania CV AB, 2016). So I treated accuracy as only a basic check. The more important parts were recall, false negatives, F1 score, and the official Scania cost.

1.2 Dataset Description

The dataset used in this project is the APS Failure at Scania Trucks dataset. It was created by Scania CV AB and used for the 2016 Industrial Challenge at the International Symposium on Intelligent Data Analysis (Lindgren & Biteus, 2016). The data came from heavy Scania trucks in normal use. The features are anonymized, so I could not connect a feature name to a specific physical sensor, but the dataset still gives enough information to train classification models.

The files already come with a training set and a test set, so I used that split instead of making my own. The training file has 60,000 rows and the test file has 16,000 rows. The full dataset has 171 columns. One column is the target called class, and the other 170 columns are numerical operational features. The class column has two values: neg for failures not related to the APS and pos for APS-related failures. Missing values are written as na in the original files.

The training set is very imbalanced. It has 59,000 negative rows and 1,000 positive rows, which is a 59 to 1 ratio. The test set has the same basic issue, with 15,625 negative rows and 375 positive rows. That class imbalance affected almost every choice in the project, especially the choice of metrics.

1.3 Objectives

The main objective was to figure out which classifier is most useful for this kind of imbalanced maintenance problem. I used these questions to guide the project:

- Which model family works best for this data: linear models, distance-based models, tree models, or ensemble models?
- How much does class weighting help models catch the positive APS-failure class?
- Can Random Forest and XGBoost beat the simpler models on F1 score?
- Does the model with the best F1 score also have the lowest official Scania cost?

These questions matter because a model can look good on paper and still be a poor maintenance tool. If it misses too many actual APS failures, the cost becomes high. If it flags too many trucks, it may waste shop time. The main thing I wanted to compare was the difference between a good machine learning score and a good practical result.

1.4 Report Structure

Section 2 goes through the exploratory data analysis. Section 3 explains the preprocessing, the models, the tuning setup, and the metrics. Section 4 compares the results and explains the tradeoff between F1 score and cost. Section 5 gives the main takeaways and possible future work.

2. Exploratory Data Analysis

Before training models, I looked through the training data to see what problems were actually there. I checked the class balance, missing values, feature distributions, outliers, and correlations. This step mattered because the cleaning choices needed to come from the data instead of being guessed ahead of time.

2.1 Class Distribution

The first thing I checked was the class distribution. It showed a major imbalance between the negative and positive classes. In the training set, 59,000 rows are negative and only 1,000 rows are positive. That means a model could predict negative for every truck and still appear to do well by accuracy.

Figure 1: Class distribution in the training set, showing the severe imbalance between negative and positive classes.

The main point from Figure 1 is that accuracy cannot be trusted by itself. This is a well known issue with imbalanced classification problems where the majority class dominates the score (He & Garcia, 2009). I still reported accuracy, but I did not treat it as the main measure. Recall matters because it shows how many real APS failures were caught. F1 score also matters because it balances recall with precision.

2.2 Missing Values

Missing values were another major issue. Some columns had so many missing values that keeping them did not seem useful. The highest-missing features included br_000 at about 82.1%, bq_000 at about 81.2%, bp_000 at about 79.6%, bo_000 at about 77.2%, ab_000 and cr_000 at about 77.2%, and bn_000 at about 73.3%. These were the columns that ended up being dropped.

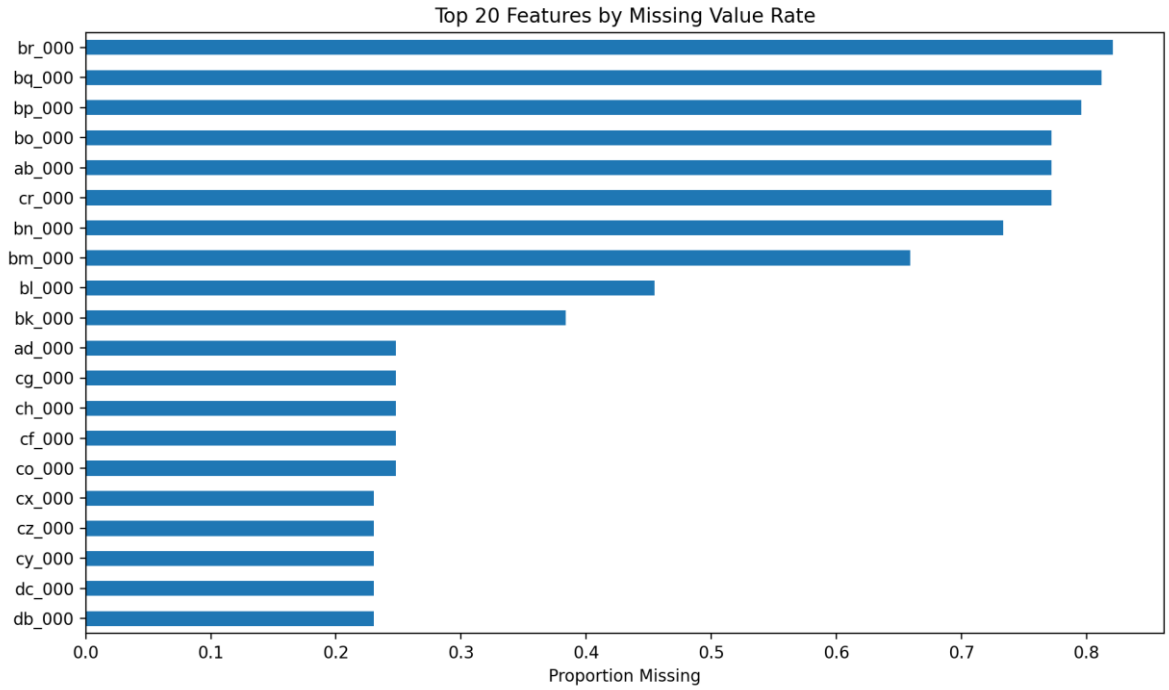

Figure 2: Top 20 features ranked by proportion of missing values in the training set.

Table 1: Highest-missing features in the training data.

Feature	Missing Rate
br_000	82.11%
bq_000	81.20%
bp_000	79.57%
bo_000	77.22%
ab_000	77.22%
cr_000	77.22%
bn_000	73.35%
bm_000	65.92%
bl_000	45.46%
bk_000	38.39%
ad_000	24.77%

cg_000	24.77%

The missing-value results led to two cleaning steps. First, I dropped columns with more than 70% missing values. Then I filled the remaining missing values using the median for each feature. I used the median because the data had a lot of skew and extreme values, so the mean could be pulled too far by outliers.

2.3 Feature Distributions

The feature histograms showed that many variables were heavily right-skewed. A lot of features had many values near zero and a smaller number of very large values. That makes sense for this type of operational data because some events may not happen often, while other trucks may record large counts.

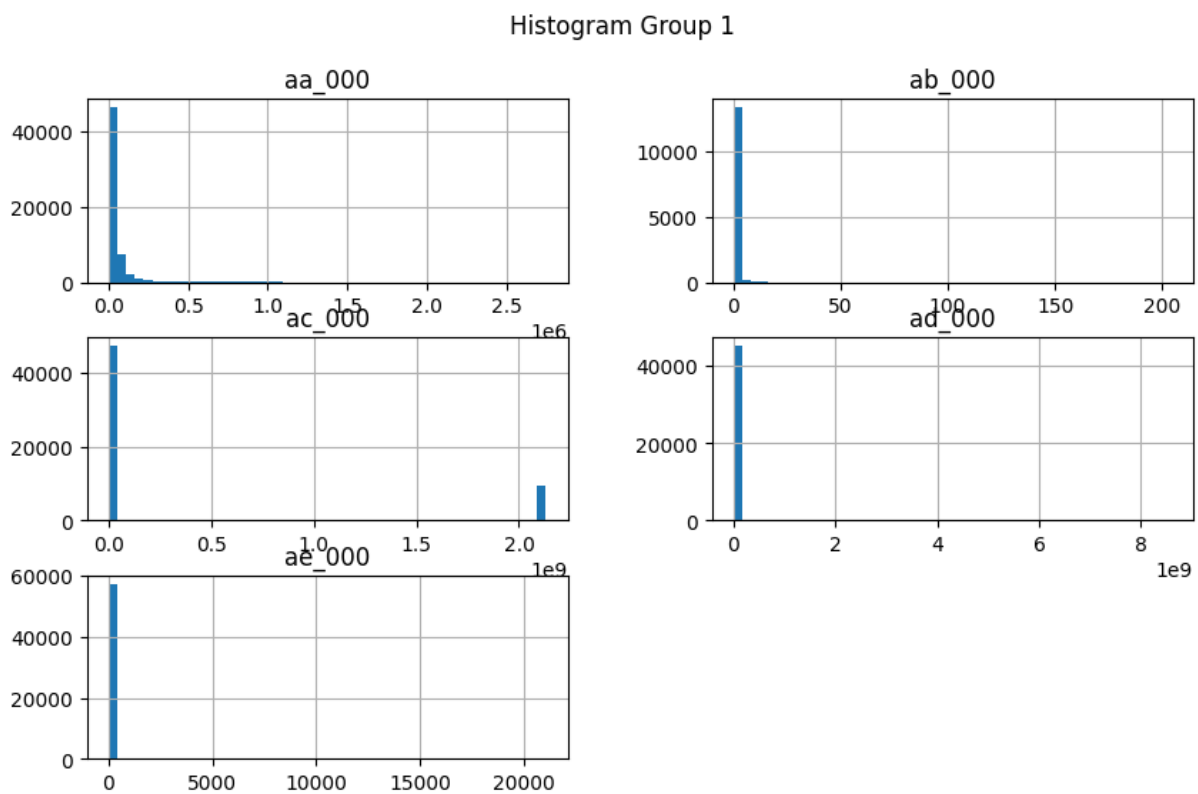

Figure 3: Histograms for a representative subset of features, showing skewed distributions and extreme values.

The feature scales were also very different from each other. Some variables had values in the millions, while others were mostly small numbers. Without scaling, models like KNN, Logistic Regression, and Linear SVM could be affected too much by whichever feature had the largest numeric range.

2.4 Outliers

The boxplots showed outliers in many features. In some plots, most of the data was squeezed near the bottom because a few values were extremely large. I did not automatically treat those

values as errors because extreme readings could still be real truck behavior. But they did affect how I handled imputation and scaling.

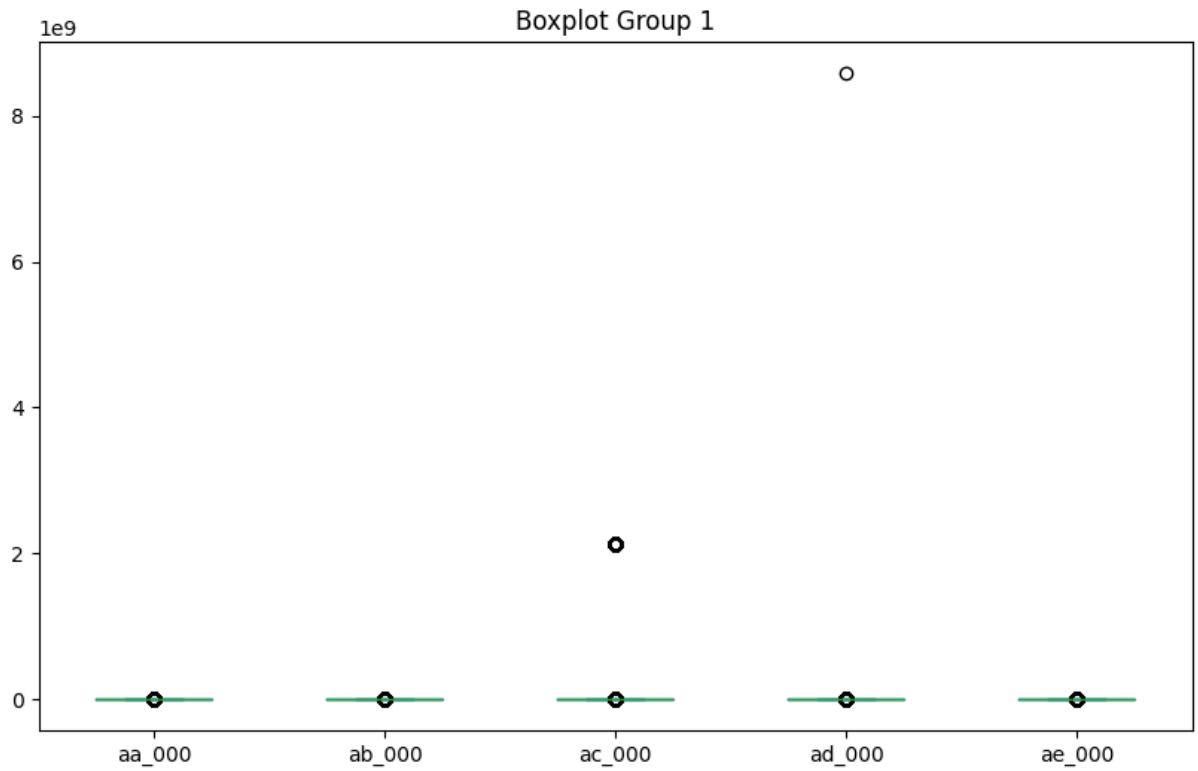

Figure 4: Boxplots for a representative subset of features, showing outliers across many variables.

I did not remove outliers from the dataset. Instead, I used median imputation and scaling. Median imputation is less sensitive to extreme values than mean imputation. Scaling also helped make the feature ranges more comparable for the models that are sensitive to magnitude. Tree-based models were less affected by scale, but I kept the preprocessing consistent where it was needed.

2.5 Feature Correlations

I also made a correlation heatmap for the first 50 feature columns. The heatmap showed some groups of related variables, especially in the earlier feature ranges. I did not see one clear block where most of the features were basically duplicates of each other, so I did not make PCA part of the main experiment.

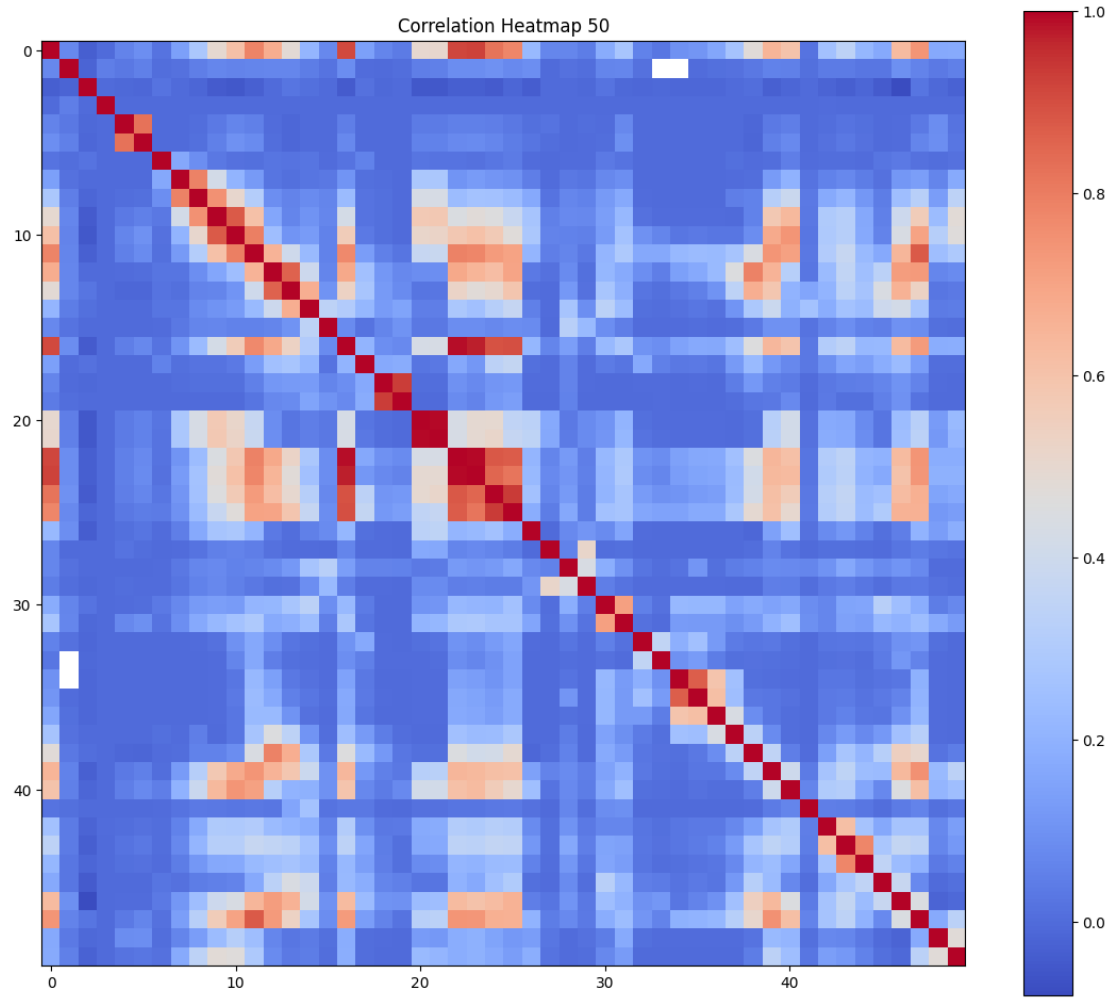

Figure 5: Correlation heatmap for the first 50 feature columns in the training set.

The correlation check did not force me to drop features. PCA could still be worth testing later because the dataset has a lot of columns. For this report, I kept the cleaned original features so the model comparison stayed easier to follow.

2.6 Key Findings from EDA

The EDA changed how I set up the rest of the project. The class imbalance pushed me toward recall, F1 score, and the cost metric. The missing-value check showed that some columns needed to be dropped. The skewed distributions supported median imputation. The scale differences showed why standardization was needed for several of the models.

- The training target was severely imbalanced: 59,000 negative cases and 1,000 positive cases.
- Seven feature columns were dropped because they had more than 70% missing values: br_000, bq_000, bp_000, bo_000, ab_000, cr_000, bn_000.
- Most features were numerical, right-skewed, and affected by extreme values.
- Feature scales varied widely, which made scaling important for linear and distance-based models.

- Correlations were present, but not strong enough to make feature reduction mandatory for the main experiment.

3. Methodology

3.1 Data Preprocessing

For the methodology, I tried to keep the comparison fair. I used the official training and test files as they were provided. I fit the preprocessing steps only on the training data and then applied the same steps to the test data. That way, the test set stayed separate until the final evaluation.

3.1.1 Handling High-Missingness Columns

Columns with more than 70% missing values were removed before imputation. This removed seven columns: br_000, bq_000, bp_000, bo_000, ab_000, cr_000, and bn_000. I chose this cutoff because a column that is missing most of the time would mostly be made up by imputation. That can make the model learn from artificial values instead of real measurements.

3.1.2 Imputation

After those columns were removed, the remaining missing values were filled with the median value for each feature. I chose the median because the EDA showed skewed distributions and outliers. In that situation, the median is usually a better simple fill value than the mean.

3.1.3 Feature Scaling

After imputation, I used StandardScaler to standardize the feature matrix. This changes each feature so it is centered around 0 with a standard deviation of 1 based on the training data. This matters for Logistic Regression, Linear SVM, and KNN because those models can be influenced by feature scale.

3.1.4 Class Imbalance Handling

I did not use oversampling or undersampling in the main experiment. Instead, I used class_weight="balanced" for the models that supported it. That setting gives more weight to the minority positive class during training, which is one of the standard cost-sensitive approaches for imbalanced data (He & Garcia, 2009). Logistic Regression, Decision Tree, Linear SVM, and Random Forest used class weighting. KNN does not support class weighting in the same way, and XGBoost used scale_pos_weight instead.

3.2 Machine Learning Models

I trained and compared six machine learning models. I picked them so the project would cover several model families and also meet the requirement to include Random Forest and XGBoost.

Logistic Regression was my linear baseline. It predicts the probability that a row belongs to the positive class using a weighted combination of the features (Müller & Guido, 2016). I included it because it is simple, fast, and works well with class weighting.

K-Nearest Neighbors was used as a model that classifies a new row by looking at nearby training rows. I included it because it is a different kind of classifier from the others, but I expected it might struggle because the data has many features and the positive class is rare.

Decision Tree was used as a single tree model. It splits the data using feature thresholds. This can capture non-linear patterns, but a single tree can also overfit, especially when the data is noisy or imbalanced.

Linear SVM was used as another linear model. It tries to separate the classes with a decision boundary that leaves as much room as possible between the two classes. Like Logistic Regression, it can use class weighting, so it was useful for checking whether a different linear method would also produce high recall.

Random Forest was used as an ensemble of decision trees. It trains many decision trees and combines their predictions. I expected it to be more stable than a single Decision Tree because averaging across trees usually reduces overfitting.

XGBoost was used as a boosted tree ensemble. It builds trees one after another, with each tree trying to fix mistakes from the earlier trees. I included it because boosted trees often work well on tabular data, and the assignment specifically required XGBoost as one of the ensemble methods.

3.3 Experimental Setup

The official Scania split was used for the experiment. The 60,000-row training file was used for preprocessing, cross-validation, and model training. The 16,000-row test file was held back for the final results. I did the tuning only inside the training set so the test results would be a fair check.

The project was implemented in Python. I used pandas and numpy for data handling, matplotlib for plots, scikit-learn (scikit-learn Developers, n.d.) for preprocessing, training, tuning, and metrics, and xgboost (XGBoost Developers, n.d.) for the XGBoost model. I used random_state=42 when the model supported it so the results would be easier to reproduce.

3.4 Hyperparameter Tuning

I used GridSearchCV with five-fold cross-validation and scoring="f1". I chose F1 for tuning because accuracy would be too easy to inflate on this dataset. F1 is not perfect, but it at least forces the model to care about both precision and recall.

Table 2: Hyperparameter grids and best cross-validated F1 scores.

Model	Parameter Grid	Best Parameters	Best CV F1
Logistic Regression	C: 0.01, 0.1, 1, 10	C = 0.01	0.5567
KNN	neighbors: 3, 5, 7, 11; weights: uniform, distance	neighbors = 3; weights = distance	0.6926
Decision Tree	max_depth: 5, 10, 20, None; min_split: 2, 10, 30	max_depth = None; min_split = 2	0.6071
Linear SVM	C: 0.01, 0.1, 1, 10	C = 0.01	0.5487
Random Forest	trees: 100, 200; max_depth: 10, 20, None	trees = 100; max_depth = None	0.7025

XGBoost	trees: 100, 200; depth: 3, 6, 10; lr: 0.1, 0.3	trees = 200; depth = 10; lr = 0.3	0.8264

The cross-validation results already suggested that XGBoost was going to be strong. Random Forest and KNN also had good cross-validated F1 scores. Still, I did not want to stop there because the final test set, confusion matrices, and cost metric told a more complete story.

3.5 Evaluation Metrics

I used five main measures: accuracy, precision, recall, F1 score, and the official Scania cost metric. I also looked at confusion matrices for every model because the raw false-positive and false-negative counts matter in this problem.

Accuracy measures how many predictions are correct out of all predictions. It is useful as a quick check, but it can be misleading here because almost every row is negative.

Precision measures how many predicted positives were actually positive. In this project, low precision means the model sends too many trucks for APS inspection. That relates to the false-positive cost of 10.

Recall measures how many real APS failures the model catches. This was the most important standard metric for the project because a missed APS failure costs 500. That is 50 times larger than the false-positive cost.

F1 score combines precision and recall into one score. I used it because it gives a better balance than accuracy on imbalanced data. XGBoost ended up with the best F1 score.

The Scania cost metric was calculated as 10 times the number of false positives plus 500 times the number of false negatives. This metric mattered because it matched the actual cost setup from the dataset. Under this formula, reducing missed failures usually matters more than reducing extra inspections.

4. Results and Discussion

4.1 Model Comparison

Table 3 shows the final test results for all six tuned models. FN means false negatives, which are missed APS failures. FP means false positives, which are unnecessary APS inspections. The final column applies the official cost formula.

Table 3: Final held-out test performance for all tuned models.

Model	Accuracy	Precision	Recall	F1 Score	FN	FP	Scania Cost
Logistic Reg.	0.9762	0.4958	0.9360	0.6482	24	357	15570
KNN	0.9884	0.8682	0.5973	0.7077	151	34	75840
Decision Tree	0.9860	0.7459	0.6107	0.6716	146	78	73780
Linear SVM	0.9764	0.4986	0.9333	0.6500	25	352	16020
Random Forest	0.9892	0.9471	0.5733	0.7143	160	12	80120
XGBoost	0.9949	0.9349	0.8427	0.8864	59	22	29720

The table shows two different ways to read the results. If I only use the usual machine learning metrics, XGBoost is the best model. It had the highest accuracy, the highest F1 score, and the best ROC-AUC. It also had very few false positives.

The cost metric changes the recommendation. Logistic regression had lower precision than XGBoost, but it missed only 24 APS failures. XGBoost missed 59. Since each missed failure costs 500, the extra false positives from logistic regression were cheaper than the extra missed failures from XGBoost.

This is the main result of the project. XGBoost is the best balanced classifier, but Logistic Regression is the lowest-cost model under the official Scania metric. A better way to put it is that the "best" model depends on what the model is being used for.

Table 4 shows the same metrics for both the training set and the test set. This comparison is useful because it shows whether each model generalized well or just memorized the training data. A large gap between the train and test scores points to overfitting.

Table 4: Training and test set performance for all tuned models.

Model	Train Acc	Test Acc	Train Prec	Test Prec	Train Rec	Test Rec	Train F1	Test F1
Logistic Reg.	0.9763	0.9762	0.4090	0.4958	0.9480	0.9360	0.5714	0.6482
KNN	1.0000	0.9884	1.0000	0.8682	1.0000	0.5973	1.0000	0.7077
Decision Tree	1.0000	0.9860	1.0000	0.7459	1.0000	0.6107	1.0000	0.6716
Linear SVM	0.9760	0.9764	0.4061	0.4986	0.9490	0.9333	0.5688	0.6500
Random Forest	1.0000	0.9892	1.0000	0.9471	0.9990	0.5733	0.9995	0.7143
XGBoost	1.0000	0.9949	1.0000	0.9349	1.0000	0.8427	1.0000	0.8864

Table 4 reveals a clear pattern. The two linear models, Logistic Regression and Linear SVM, did not overfit. Their training and test scores are almost identical, which means the patterns they learned during training generalized to new trucks. The four non-linear models all reached perfect or near-perfect training scores, but their test scores dropped significantly. KNN, Decision Tree, and Random Forest each had training F1 scores at or near 1.0, while their test F1 scores fell to

between 0.67 and 0.71. That gap is a sign that those models memorized the training data instead of learning a general rule.

XGBoost also reached a training F1 of 1.0, but its test F1 stayed strong at 0.8864. That means XGBoost still overfit, but it generalized much better than the other tree-based models. This is one of the reasons XGBoost ended up with the best balanced classification performance: even when it memorized the training data, the boosting process produced a model that still worked well on unseen examples.

4.2 Performance Visualization

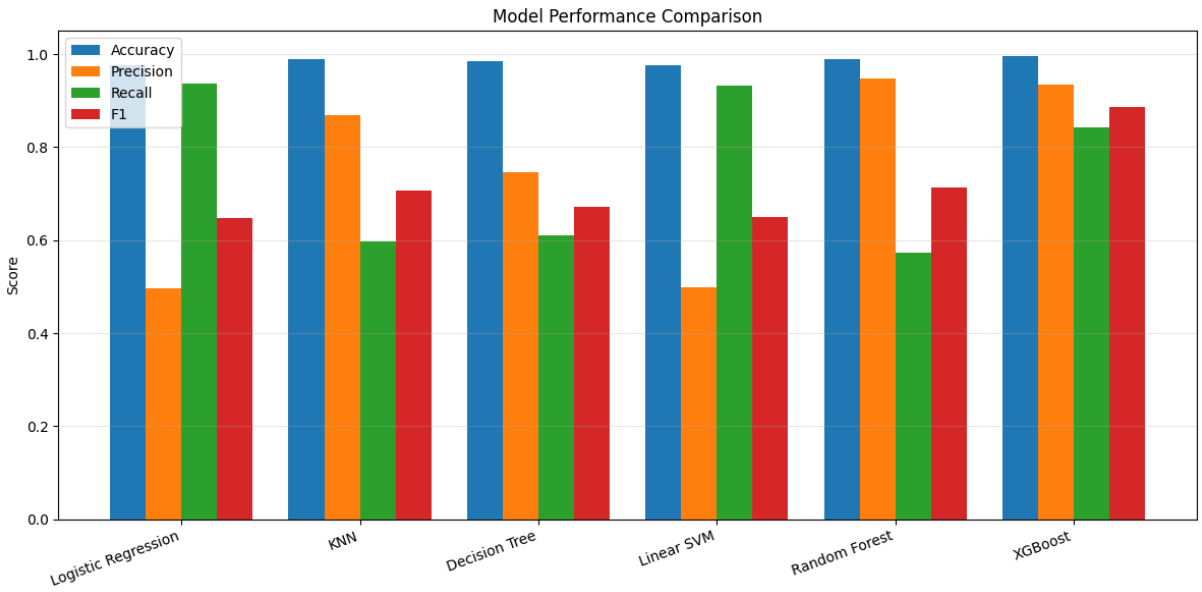

Figure 6: Comparison of accuracy, precision, recall, and F1 score across all six tuned models.

Figure 6 makes the precision and recall tradeoff easier to see. Logistic Regression and Linear SVM had the highest recall, but their precision was much lower. KNN, Decision Tree, and Random Forest were more precise, but they missed too many failures. XGBoost was the model that stayed strong across most of the standard metrics.

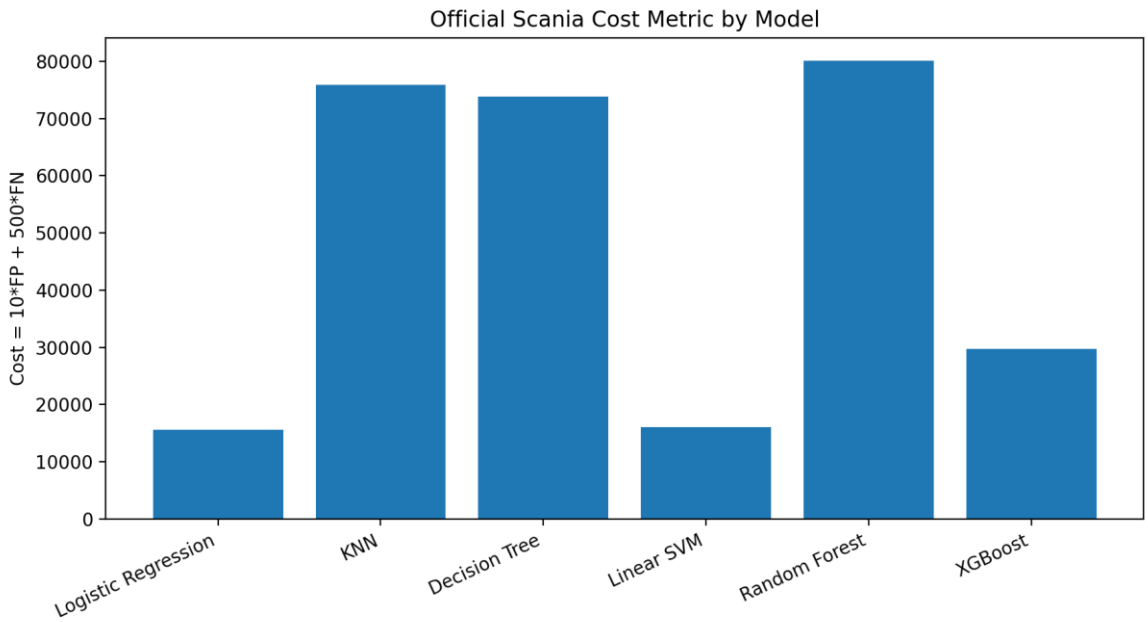

Figure 7: Model comparison using the official Scania cost metric.

Figure 7 shows why the cost metric matters. Random Forest had very high precision, but it had the worst cost because it missed 160 actual APS failures. Logistic Regression had many false positives, but the dataset treats those as much cheaper mistakes. That is why Logistic Regression became the lowest-cost model.

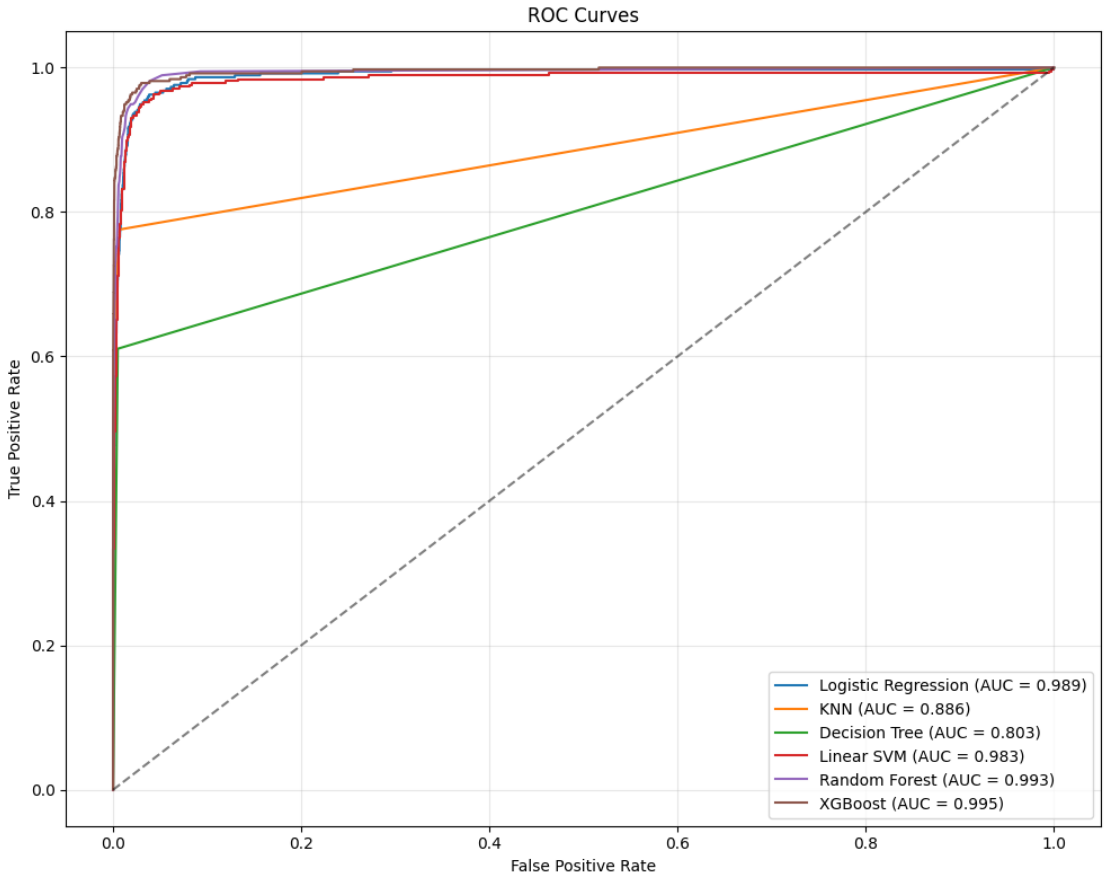

Figure 8: ROC curves for all six tuned models on the held-out test set.

The ROC curves show that several models were able to separate the two classes fairly well. XGBoost had the strongest AUC at about 0.995, and Random Forest was close behind at about 0.993. Logistic Regression and Linear SVM were also strong at around 0.98. This made me think threshold tuning could help because the models may have useful probability scores even if the default decision cutoff is not ideal.

4.3 Best Model Analysis

There are two reasonable ways to name the best model. By F1 score, accuracy, and ROC-AUC, XGBoost is the best model. It reached a 0.8864 F1 score, 0.9349 precision, and 0.8427 recall. That makes it the strongest general classifier in the experiment.

By the official Scania cost metric, Logistic Regression is the best model. It caught 351 of the 375 APS failures in the test set and missed only 24. It also created 357 false positives, which is not ideal, but those mistakes cost much less than false negatives in this dataset.

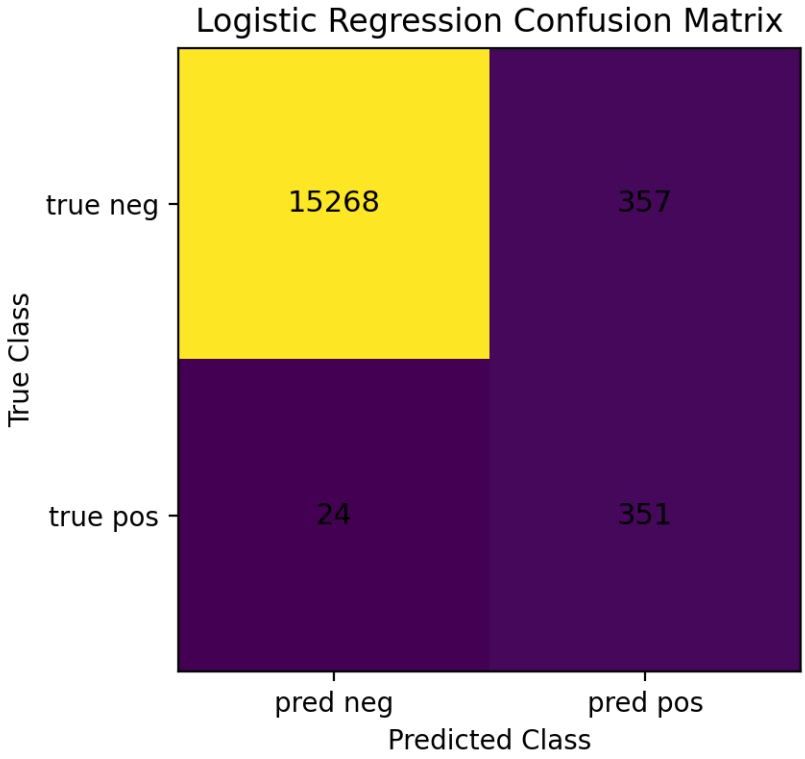

Figure 9: Confusion matrix for Logistic Regression, the lowest-cost model under the official Scania metric.

The Logistic Regression confusion matrix shows why it performed well on cost. It produced 351 true positives and only 24 false negatives. The weakness is the 357 false positives. In a real shop, that means a lot of extra inspections. But according to the dataset cost formula, those extra inspections are still cheaper than missing more real failures.

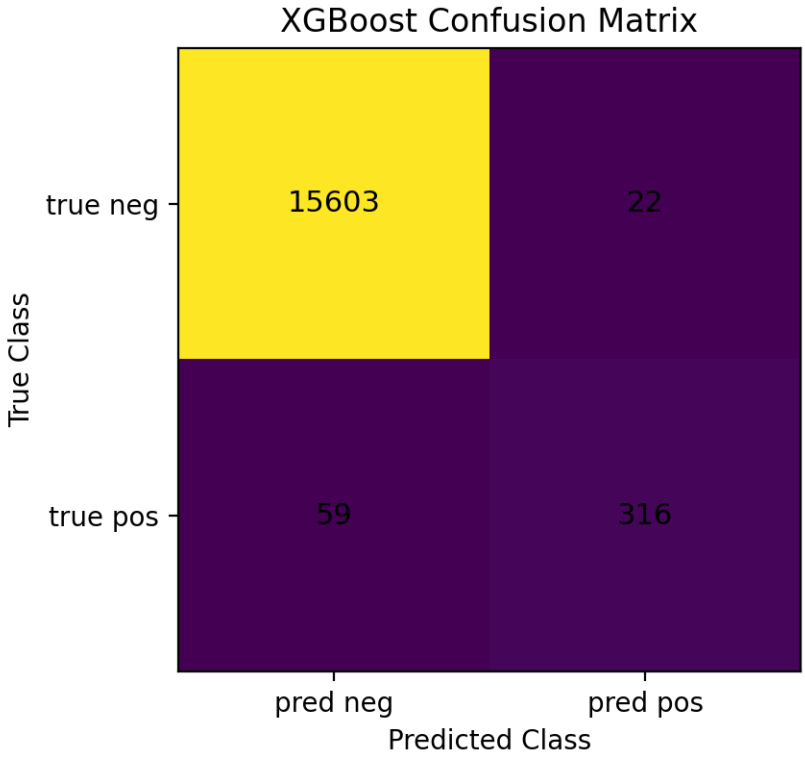

Figure 10: Confusion matrix for XGBoost, the strongest model by F1 score.

The XGBoost confusion matrix shows a different strength. It produced only 22 false positives, so it was much more careful about flagging trucks as APS failures. It also caught 316 of the 375 real failures. The problem is that it missed 59 failures, which is more than twice the number missed by Logistic Regression.

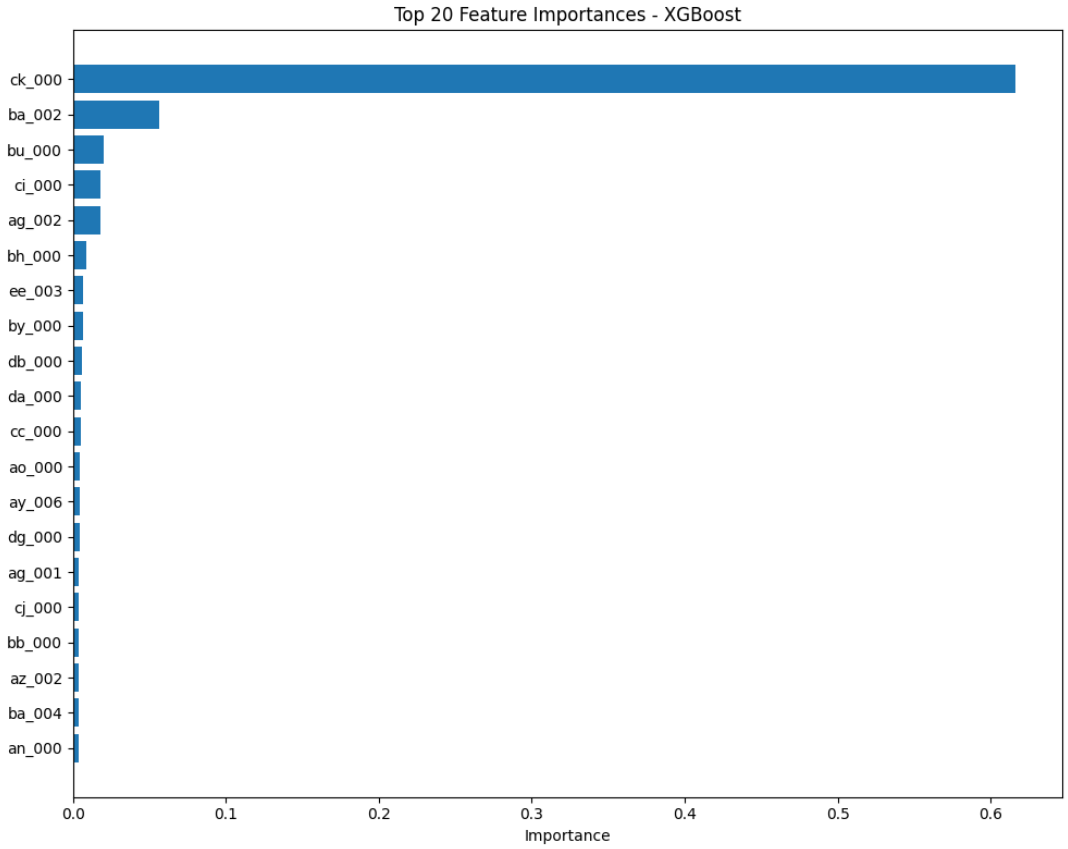

Figure 11: Top 20 feature importances from the XGBoost classifier.

The XGBoost feature-importance plot showed that one anonymized feature, ck_000, had the largest influence on the model. The next most important feature was ba_002, followed by several smaller contributors. Since the feature names are anonymized, I cannot say exactly what truck measurement those features represent. Still, the plot suggests that a smaller group of variables may be carrying a lot of the signal.

4.4 Discussion

The results showed that class imbalance affected the model families in different ways. Logistic Regression and Linear SVM were the most aggressive about predicting the positive class. Class weighting helped push them toward higher recall. That helped them catch more APS failures, but it also created more false positives.

KNN, Decision Tree, and Random Forest had the opposite problem. They were more selective when they predicted the positive class, so their precision was better. But they missed many more real APS failures. KNN probably struggled because the dataset is high-dimensional and dominated by negative examples. Which makes sense since distance-based methods tend to break down with that many features. The tree models handled scaling and outliers better, but they still did not catch enough positive cases.

XGBoost did better than the other non-linear models because boosting keeps correcting earlier mistakes. The scale_pos_weight setting also gave the positive class more influence during

training. That definitely helped XGBoost catch more failures than Random Forest while keeping false positives low.

The main interpretation is that model performance depends on the goal. XGBoost looked best from a standard ML point of view because it balanced precision and recall. Logistic Regression looked best from a maintenance-cost point of view because it missed fewer APS failures. These two results do not cancel each other out. They answer different versions of the same question.

The training versus test comparison in Table 4 also helps explain why the models behaved the way they did. The two linear models did not overfit. Their training and test scores were almost the same, which means they actually generalized. KNN, Decision Tree, and Random Forest all reached perfect or near-perfect training scores but dropped significantly on the test set, especially on recall. That gap explains why those models missed so many real APS failures even after class weighting. They had memorized the training data, including the small number of positive examples, but they could not extend that pattern to new trucks. XGBoost also overfit on training, but its test F1 stayed strong at 0.8864, which is much closer to its training score than for Random Forest. This is one reason XGBoost still ended up as the strongest balanced model on the test set even though all four non-linear models showed signs of overfitting.

The main limitation is that the models used their default classification thresholds. In many classifiers, a probability above 0.5 becomes a positive prediction. That may not be the right cutoff here because false negatives cost 50 times more than false positives. A better follow-up would tune the threshold directly against the Scania cost metric instead of using the default cutoff.

5. Conclusion

5.1 Summary of Findings

This project compared six machine learning classifiers on the APS Failure at Scania Trucks dataset. The data was challenging because it had anonymized numerical features, many missing values, skewed distributions, outliers, and a very imbalanced target. Because the positive class was rare, I had to look beyond accuracy and focus on recall, F1 score, false negatives, and cost.

The strongest model by balanced classification performance was XGBoost. It reached 0.9949 accuracy, 0.9349 precision, 0.8427 recall, and 0.8864 F1 score on the test set. It also had the strongest ROC-AUC and very few false positives. So if the goal is the best general classification performance, XGBoost is the best choice from this experiment.

The strongest model by the official Scania cost metric was logistic regression. It had lower precision, but it missed only 24 APS failures. Since false negatives are much more expensive than false positives here, Logistic Regression produced the lowest total cost. Basically, the answer depends on the goal: XGBoost is best by F1 score, while Logistic Regression is best by the cost formula.

5.2 Answers to Research Questions

Research Question 1 asked which model family worked best for the dataset and the answer to that depends on the metric. XGBoost, a boosted ensemble model, worked best for balanced prediction. Logistic Regression, a class-weighted linear model, worked best for cost because it had the fewest false negatives.

Research Question 2 asked how much class weighting helped recall. It helped the linear models the most. Logistic Regression and Linear SVM both reached recall above 0.93. The tree models did not reach that same level of recall, which shows that class weighting alone was not enough for every model type.

Research Question 3 asked whether the ensemble models could beat the simpler models on F1 score. XGBoost did beat the other models on F1 score with a score of 0.8864. Random Forest did not perform as well because it missed too many positive cases, even though it had strong precision.

Research Question 4 asked whether the best F1 model also had the lowest cost. It did not. XGBoost had the best F1 score, but Logistic Regression had the lowest Scania cost. This was probably the most useful lesson from the project because it showed that the metric changes the model choice.

5.3 Future Work

The first direction for future work is threshold tuning. Instead of using the default 0.5 cutoff, future work should test different thresholds and choose the one with the lowest Scania cost. This might let XGBoost keep its strong F1 score while reducing missed APS failures.

The second direction is to re-tune all six models using cost as the GridSearchCV scoring function instead of F1. The current setup picked the best hyperparameters for F1, then evaluated cost afterward. Tuning directly against cost would match the optimization to the metric the

dataset actually defines, and it would be a fair test of which model wins when the right metric is used from the start.

The third direction is feature selection or dimensionality reduction. PCA or model-based selection could reduce the number of features. That might make the models faster and possibly help generalization.

The fourth direction is sensitivity analysis on the cost ratio. The Scania cost values of 10 and 500 give a 50-to-1 ratio, but in a real maintenance setting that ratio could be different depending on the actual workshop and breakdown costs. Re-running the comparison at different ratios would show how robust the model rankings are, and at what point XGBoost might overtake Logistic Regression.

References

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Lindgren, T., & Biteus, J. (2016). APS Failure at Scania Trucks Data Set. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks

Müller, A. C., & Guido, S. (2016). Introduction to Machine Learning with Python. O'Reilly Media.

Scania CV AB. (2016). APS Failure and Operational Data for Scania Trucks: Dataset description and IDA 2016 Industrial Challenge cost metric.

scikit-learn Developers. (n.d.). scikit-learn: Machine Learning in Python. Retrieved from https://scikit-learn.org/stable/

XGBoost Developers. (n.d.). XGBoost Documentation. Retrieved from https://xgboost.readthedocs.io/